



Aula 03: Classes Java

▼ Classes Java

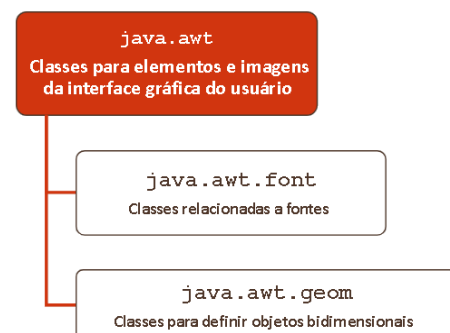
▼ Pacotes (Biblioteca de Classes Java)

As classes da biblioteca de classes Java são organizadas em pacotes
Um pacote contém um grupo de classes relacionadas

Alguns exemplos de pacotes da biblioteca de classes Java:

Pacote	Finalidade
java.lang	Fornecer classes que são fundamentais para o design da linguagem Java
javax.swing	Fornecer classes para construir componentes da interface gráfica do usuário
java.net	Fornecer classes para aplicativos de rede
java.time	Fornecer classes para datas, horas, instantes e durações

A vasta coleção de classes é organizada em uma hierarquia semelhante a uma árvore, que permite aos pacotes serem divididos em subpacotes, como este:



Para utilizar pacotes em seu programa você deve usar o comando

import :

```
import java.util.Scanner // I
mporta a classe Scanner do pa
cote java.util
import java.util.* // Importa
```

```
java.util.Scanner
      Pacote      Nome da Classe
```

```
todas as classes do pacote java.util
```

▼ String

▼ Introdução

Uma *string* é uma sequência de caracteres que inclui letras do alfabeto, caracteres especiais e espaço em branco.

Em Java, as *strings* não são um tipo de dados primitivos, invés disso elas são objetos da classe *String*.

A classe *String* é importada do pacote `java.lang.String`

As strings podem ser instanciadas como qualquer outra classe Java:

```
String str = new String("Hello world!");
```

Mas isto não é muito usual.

A forma mais comum de se instanciar uma *string* é chamada de *string* literal:

```
String str = "Hello world!";
```

Esta forma lembra a declaração de uma variável primitiva, mas lembre-se, **uma *string* é um objeto da classe *String*!**

A Classe *String* possui muitos métodos, a seguir serão apresentados alguns deles.

<i>String</i>
- data: []char
+ length(): int
+ equals(): boolean
+ indexOf(char c): int
+ charAt(int index): char
+ compareTo(String otherString): int
+ substring(int beginIdx, int endIdx): String
+ replace(char oldChar, char newChar): String

▼ Métodos da Classe String:

Método `str.length()` :

Você pode calcular o comprimento de uma string usando o método *length* definido na classe String:

- Método: `name.length()`
- Retorna o comprimento ou o número de caracteres no nome como um valor inteiro

Método `str.indexOf()` :

Você pode acessar cada caractere em uma string por seu índice numérico

- O primeiro caractere da string está no índice 0, o seguinte está no índice 1 e assim por diante
- Por exemplo:

```
String str= "Hello, World";
```

H	e	l	l	o	,		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10	11

- `str` tem de 0 a 11 índices; ou seja, entre 0 a `str.length()-1`

Cada caractere de uma *string* tem um índice. Você pode recuperar o valor do índice de um caractere na string usando o método `indexOf()` :

Método: `str.charAt(int index)` :

Retorna o caractere da string localizada no índice passado como o parâmetro

Método

`str.substring()` :

Você pode extrair uma *substring* de determinada string

- O Java fornece dois métodos para essa operação:

Método	Descrição
<code>str.substring(int beginIdx)</code>	Retorna a substring de beginIdx até o fim da string
<code>str.substring(int beginIdx, int endIdx)</code>	Retorna a substring de beginIdx até, mas não inclusive, endIdx

Método `str.replace()` :

Este método substitui todas as ocorrências dos caracteres correspondentes em uma *string*

- Método: `replace(char oldChar, char newChar)`

Método

`str.replaceFirst()` :

Este método só substitui a 1ª ocorrência do padrão de caracteres correspondentes em uma *string*

- Método: `replaceFirst(String pattern, String replacement)`

Para conhecer os demais métodos da classe String

Método	Descrição
<code>str.indexOf(char c)</code>	Retorna o valor do índice da primeira ocorrência de c na String str
<code>s1.indexOf(char c, int beginIdx)</code>	Retorna o valor do índice da primeira ocorrência de c em String s1, começando em beginIdx até o fim da string

consulte a documentação:

Java Development Kit Version 17 AF
 declaration: module: java.base, package
 class: String
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>

▼ Classe final

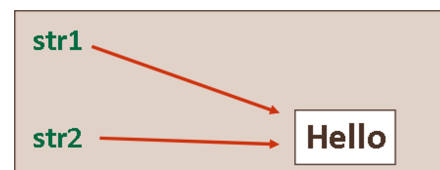
Um objeto String é imutável; ou seja, depois que um objeto String é criado, seu valor não pode ser alterado.

Como as strings são imutáveis, o Java pode processá-las de maneira muito eficiente

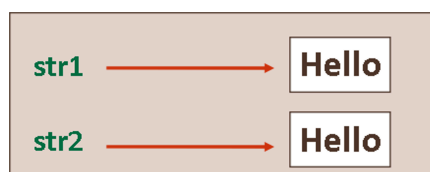
- Considere o seguinte:

```
String str1 = "Hello";
String str2 = "Hello";
```

- Mas isso é o que acontece:



- Esperamos isso:



O sistema de run-time Java sabe que as duas strings são idênticas e aloca o mesmo local de memória para os dois objetos

Maiores detalhes de Classes Finais serão abordados mais a frente na sessão **Modificador FINAL**.

▼ Concatenando Strings

No Java, a concatenação de strings forma uma nova string que é a combinação de várias strings.

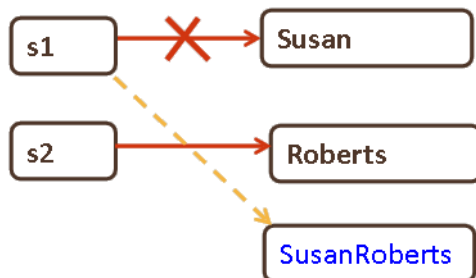
Você pode concatenar strings em Java de duas maneiras:

- operador de concatenação de strings `+`
- método `concat()`

operador +:

- Pode funcionar entre uma string e uma string ou um valor de tipo de dados char, int, double ou float
- Converte o valor em sua representação de string antes da concatenação

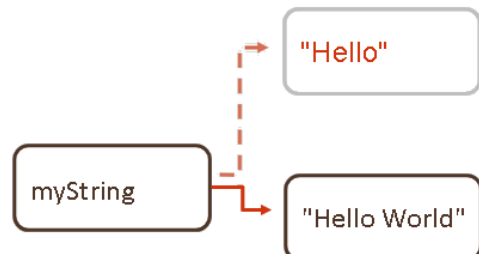
```
String s1 = "Susan";  
String s2 = "Robert  
s";  
s1 = s1 + s2;
```



Método `concat()`:

- Só pode ser chamado em strings
- Verifica a compatibilidade dos tipos de dados. Será produzido um erro de tempo de compilação se não houver compatibilidade

```
String myString = "Hello";  
myString = myString.concat(" World");
```



▼ Comparação entre Strings

Método `equals()`

- Retorna `false` se a sequência de caracteres das strings comparadas forem diferentes.
- Retorna `true` se a sequência de caracteres

Método `compareTo()`

- Retorna o valor inteiro zero se as strings comparadas forem idênticas.
- Retorna valor inteiro >0 ou <0 se as strings comparadas forem

das strings comparadas forem iguais.

- Retorna `false` se o objeto passado como parâmetro não for uma string.

```
"cab".equals("car")
//return false
"car".equals("car")
//return true
"car".equals(3) //return false
```

diferentes. O valor retornado depende da diferença lexicográfica entre as strings ou da diferença entre os tamanhos das strings. Consulte a documentação para maiores detalhes.

- Lança uma exceção se o objeto passado como parâmetro não for uma String .

```
"computer".compareTo("comparison"); //return >0
"cab".compareTo("car"); //return <0
"car".compareTo("car"); //return =0
```

▼ Random Introdução

Números aleatórios são essenciais em programas de computadores para introduzir variedade e imprevisibilidade, simulando situações reais e promovendo segurança em criptografia. Exemplos relevantes incluem:

1. **Simulações:** Para modelar sistemas complexos, como previsão do tempo, tráfego de redes, ou jogos.

Como instanciar a classe *Random*:

```
Random r = new Random();
```

Métodos da classe *Random*:

Método	Produz
<code>boolean nextBoolean();</code>	Um valor verdadeiro ou falso
<code>int nextInt();</code>	Um valor integral entre <code>Integer.MIN_VALUE</code> e <code>Integer.MAX_VALUE</code>
<code>long nextLong();</code>	Um valor longo integral entre <code>Long.MIN_VALUE</code> e <code>Long.MAX_VALUE</code>
<code>float nextFloat();</code>	Um número decimal entre 0 (inclusive) e 1 (exclusive)
<code>double nextDouble();</code>	Um número decimal entre 0 (inclusive) e 1 (exclusive)

2. **Criptografia:** Geração de chaves criptográficas, garantindo segurança em transações online.
3. **Jogos:** Sorteio de cartas, dados, ou posições em jogos eletrônicos para tornar as experiências mais dinâmicas e desafiadoras.
4. **Testes de Software:** Para criar casos de teste não determinísticos e garantir robustez em programas.
5. **Segurança:** Geração de tokens de autenticação temporários ou códigos de verificação para prevenir ataques de repetição.

Em Java, você usa a classe *Random* para obter números aleatórios.

A classe está localizada no pacote `java.util`.

Random
- seed: int
+ nextBoolean(): boolean
+ nextInt(): int
+ nextLong(): long
+ nextFloat(): float
+ nextDouble(): double
+ setSeed(int seed): void

Ela contém vários métodos que retornam um valor obtido aleatoriamente que pode ser dos tipos

`inteiro`, `double`,
`boolean`, `float` e
`long`.

Com exceção do método `nextBoolean()`, os números pseudoaleatórios gerados vão sempre de 0 (inclusive) até o valor passado como parâmetro para o método (não inclusive).

Seed

O método construtor da classe *Random* possui sobrecarga para receber um valor de *seed* como parâmetro. Este recurso é utilizado para uma sequência pseudoaleatória de números seja sempre a mesma.

Você também pode setar a semente do objeto *Random* através do método `.setSeed()`

▼ JOptionPane

▼ Introdução

A classe JOptionPane em Java é uma parte da biblioteca javax.swing que fornece caixas de diálogo pré-configuradas para interação com o usuário em aplicativos gráficos. Ela permite exibir mensagens, solicitar entrada de dados e realizar confirmações. Com métodos simples, é possível criar e personalizar diferentes tipos de caixas de diálogo, como mensagem informativa, alerta, entrada de texto, seleção de opções, entre outros. Essa classe é amplamente utilizada em interfaces gráficas de usuário (GUI) em aplicações Java para proporcionar uma experiência interativa e amigável ao usuário.

JOptionPane
+ showInputDialog(String msg): String
+ showMessageDialog(String msg): void

▼ Métodos

Método

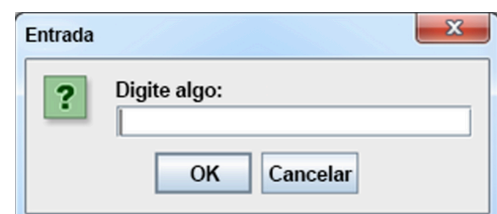
`JOptionPane.showInputDialog()` :

```
String[] acceptableValues = {"Escolha 1",  
"Escolha 2",  
"Escolha 3"};  
String input = (String) JOptionPane.showInputDialog(null,  
"Esta é uma pergunta",  
"Escolha uma opção",  
JOptionPane.DEFAULT_OPTION,  
acceptableValues,  
null,  
null);
```

Método

`JOptionPane.showInputDialog()` com mais parâmetros:

```
String input = JOptionPane.showInputDialog("Digite algo:");
```



```

a?",

"Título da Caixa de D
iálogo",

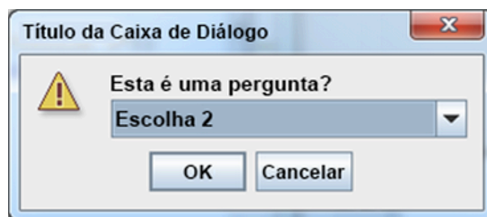
2,

null,

acceptableValues,

acceptableValues[1]);

```



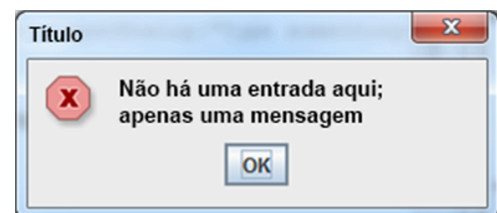
```

JOptionPane.showMessageDialog(
    null,
    "Não há u
ma entrada aqui; apen
as uma mensagem ",
    "Título",
    0);

```

Método

```
JOptionPane.showMessageDialog() :
```



Documentação APIs JAVA:

Java Development Kit Version 17 API Specification

declaration: module: java.desktop, package: javax.swing, class: JOptionPane

 <https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/javax/swing/JOptionPane.html>

▼ Classes Estáticas

```

public sta
tic class
ClasseEstá
tica{

}

```

Uma classe estática em programação orientada a objetos é uma classe que não pode ser instanciada e cujos membros (métodos e variáveis) pertencem à classe em si, não a instâncias individuais dela.

Isso significa que você pode acessar os membros estáticos diretamente usando o nome da classe, sem precisar criar um objeto dela.

Exemplo:

▼ Math

A classe Math contém métodos para realizar operações numéricas básicas, como funções exponenciais elementares, logaritmos, raízes quadradas e trigonométricas.

A classe Math é uma classe estática, o que significa que você não precisa instanciar para utilizá-la.

Após importá-la seus métodos podem ser acessados diretamente:

```
import java.lang.Math

public class Main {
    public static void
    main(String[] args) {
        System.out.prin
        tln(Math.sqrt(4));
    }
}
```